

M07 Teaching Book

Beginner-first, self-contained PostgreSQL + Data Modeling. External media helps visualization; this book remains sufficient to recover the competency when a video changes or disappears.

VIDEO FIRST -> close video -> SOLO TEACHING BOOK -> guided reproduction -> independent task -> validate result -> explain in own words -> protected Review only after genuine saved attempt -> fresh changed retry if weak -> evidence/mastery

DE07-01 - Verify PostgreSQL connection and database/schema context

VIDEO FIRST route

[freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners | 0:30:15-0:38:12 - create/connect database + safety orientation](#)

Starting state / prerequisite

M03 PostgreSQL mini-lab completed; M06 terminal basics available.

Outcome

Prove the active database, role and schema search context before changing any object.

Teach it from zero

A PostgreSQL server (cluster) can contain several databases. A client connection works inside one database at a time, and that database contains schemas. Unqualified table names are resolved through `search_path`. This is why a correct statement against the wrong database or schema is still a serious mistake. M07 reuses the PostgreSQL installation introduced earlier; do not reinstall simply because this module begins.

Teacher demonstration

```
SELECT current_database() AS db, current_user AS db_user, current_schema() AS current_schema;  
SHOW search_path;  
SELECT version();
```

Guided reproduction

Connect to the dedicated stage07_de database. Run the context queries, then create no objects until the database/user/search path evidence is saved.

Independent task

Close video/book. Reconnect in a fresh query tab or psql session and prove context again. Explain why the GUI tree selection alone is insufficient evidence.

Validate

`current_database()` is stage07_de; `current_user` is known; `search_path` is recorded; no destructive statement ran before context proof.

Recover

If context is wrong, stop. Reconnect to stage07_de rather than editing setup SQL to fit the wrong database.

Explain back

Explain verify postgresql connection and database/schema context in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Schemas](#)

[PostgreSQL 18 - psql](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-02 - Tables, data types and explicit DDL

VIDEO FIRST route

[freeCodeCamp - PostgreSQL tables and constraints | 0:41:37-0:55:55 - CREATE TABLE, table structure and constraints](#)

Starting state / prerequisite

DE07-01 mastered.

Outcome

Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL.

Teach it from zero

Types are part of the data contract. Use date for calendar dates, timestamptz when an instant/time-zone meaning matters, integer/bigint for whole numbers, numeric(p,s) when exact decimal precision matters, boolean for true/false, and text for variable text. DEFAULT supplies a value when a column is omitted; it does not magically replace an explicitly supplied NULL. DDL should live in scripts so a second learner can reproduce the schema.

Teacher demonstration

```
CREATE TABLE source.suppliers (  
  supplier_id text PRIMARY KEY,  
  supplier_name text NOT NULL,  
  country text NOT NULL,  
  active boolean NOT NULL DEFAULT true,  
  rating numeric(3,2) CHECK (rating BETWEEN 0 AND 5)  
);
```

Guided reproduction

Create source.suppliers from the course specification, then inspect information_schema.columns instead of trusting the GUI alone.

Independent task

Create source.carriers with a text business key, required name, active boolean default, joined_on date and exact rating/score field. Justify each type in comments.

Validate

Column definitions match the written contract; dates/numbers are not stored as text merely to avoid parsing; DDL is saved.

Recover

If a value does not fit the type, decide whether the data is invalid or the type contract was wrong. Do not weaken the type automatically.

Explain back

Explain tables, data types and explicit ddl in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Data Types](#)

[PostgreSQL 18 - CREATE TABLE](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-03 - Primary keys, foreign keys and constraints

VIDEO FIRST route

[freeCodeCamp - PostgreSQL keys and constraints | 2:29:24-2:54:45 + 3:16:41-3:25:04 - primary/unique/check/foreign keys](#)

Starting state / prerequisite

DE07-02 mastered.

Outcome

Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges.

Teach it from zero

PRIMARY KEY identifies a row and is unique/non-null. UNIQUE protects alternate keys. FOREIGN KEY requires the referenced parent key to exist. NOT NULL makes presence mandatory. CHECK enforces a row-level predicate. Constraint failures during a controlled test are good evidence: they prove the database boundary is working. Never remove a valid integrity rule simply to make deliberately bad practice data load.

Teacher demonstration

```
INSERT INTO source.suppliers VALUES ('S001', 'Demo', 'IN', true, 4.50);  
-- deliberate failures, one at a time:  
-- duplicate S001; NULL supplier_name; rating 6.00
```

Guided reproduction

Run one valid supplier insert, then one duplicate-key, one NULL-required-value and one out-of-range test. Save each error and map it to the constraint.

Independent task

Design a parent carrier table plus a child route table. Prove one orphan route is rejected by the FK and roll back/clean only the test state.

Validate

Each invalid state is rejected for the expected reason; valid rows remain; no constraint was disabled/deleted as a shortcut.

Recover

Read the exact violated constraint and identify the business rule. Repair the bad input or model—not the evidence.

Explain back

Explain primary keys, foreign keys and constraints in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Constraints](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-04 - Transactions, DML and UPSERT

VIDEO FIRST route

[Software Nuggets - PostgreSQL Transactions | 0:40-6:47 - transaction overview, COMMIT and ROLLBACK; then freeCodeCamp 3:01:36-3:16:41 for UPDATE/ON CONFLICT/UPSERT](#)

Starting state / prerequisite

DE07-03 mastered.

Outcome

Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule.

Teach it from zero

BEGIN starts an explicit transaction, COMMIT makes its changes durable, and ROLLBACK discards uncommitted changes. The strongest rollback proof records before, inside-transaction and after-rollback states. UPSERT is not "update anything that conflicts"; ON CONFLICT must target a meaningful unique key and define which attributes are safe to update. Transactions preserve atomicity boundaries but cannot make incorrect business logic correct.

Teacher demonstration

```
BEGIN;  
UPDATE source.customers SET region='South' WHERE customer_id='C001';  
SELECT customer_id,region FROM source.customers WHERE customer_id='C001';  
ROLLBACK;  
SELECT customer_id,region FROM source.customers WHERE customer_id='C001';
```

Guided reproduction

Run the rollback demonstration, then perform one controlled UPSERT on an existing supplier or dimension row.

Independent task

Change C002 inside a new transaction, prove temporary state, roll back, then write an UPSERT for a different key with a comment explaining conflict policy.

Validate

Before/inside/after evidence proves rollback; post-rollback source state matches baseline; UPSERT target is a real unique key and only intended fields update.

Recover

If a GUI autocommits unexpectedly, stop and reproduce in one explicit transaction block. A COMMIT already issued cannot be undone by ROLLBACK.

Explain back

Explain transactions, dml and upsert in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Transactions tutorial](#)

[PostgreSQL 18 - INSERT / ON CONFLICT](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-05 - Indexes and EXPLAIN evidence

VIDEO FIRST route

[Milan Jovanovic - How SQL Indexes Actually Work \(PostgreSQL\) | Whole focused video - B-tree/index choice + EXPLAIN ANALYZE; do not memorize its timing numbers](#)

Starting state / prerequisite

DE07-04 mastered.

Outcome

Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data.

Teach it from zero

An index is a separate structure maintained as table rows change, so it trades storage/write work for potential read efficiency. Index the access pattern, not every column. EXPLAIN shows the planner estimate; EXPLAIN ANALYZE executes and adds actual evidence, so use it carefully. On tiny course tables a sequential scan may remain cheaper even after a sensible index exists. The lab is about evidence, not forcing a plan shape.

Teacher demonstration

```
EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003';  
CREATE INDEX IF NOT EXISTS idx_orders_customer_id ON source.orders(customer_id);  
EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003';
```

Guided reproduction

Capture before/after plans for the customer lookup and write only what actually changed. Record that Seq Scan on tiny data is acceptable.

Independent task

Choose a different repeated filter/join from the lab, propose one index, capture before/after EXPLAIN and state one write/storage cost.

Validate

Claim is supported by observed plan evidence; no “index always faster” statement; index matches a real predicate/join/order pattern.

Recover

If the planner still uses Seq Scan, do not force an index. Explain table size/selectivity and keep the evidence.

Explain back

Explain indexes and explain evidence in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Indexes](#)

[PostgreSQL 18 - Using EXPLAIN](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-06 - Operational vs analytical modeling and normalization

VIDEO FIRST route

[Maven Analytics - Database Normalization for Beginners | 0:13-1:56 normalization meaning + 10:57-18:00 2NF/3NF/pros-cons: use 1:56-9:36 only as relational/star visual support](#)

Starting state / prerequisite

DE07-05 mastered.

Outcome

Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation.

Teach it from zero

Operational models usually separate business entities so one fact or attribute has an authoritative home and updates do not require changing dozens of copied values. Analytical models often reorganize data around measurable events and descriptive dimensions. Normalization is not a moral score: the right boundary follows workload, ownership, integrity and grain. Before deciding table boundaries, state what one row means and which business key identifies it.

Teacher demonstration

Operational: customers -> orders -> order_items. Customer name/region live once in customers. Analytical: fact_sales repeats di

Guided reproduction

Inspect source.customers/orders/order_items and list three update/anomaly problems that would appear if customer_name and region were copied into every order line.

Independent task

Take a wide carrier-delivery row and sketch an operational decomposition. Then state which repeated keys would be acceptable in an analytical fact.

Validate

You distinguish harmful duplicated attributes from intentional repeated foreign keys at fact grain; every proposed table has a written grain.

Recover

If you are arguing "normalized good / denormalized bad," return to workload + integrity + grain and explain the trade-off.

Explain back

Explain operational vs analytical modeling and normalization in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - DDL basics](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-07 - Facts, dimensions and star-schema grain

VIDEO FIRST route

[Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - overview, facts, dimensions, marts and benefits](#)

Starting state / prerequisite

DE07-06 mastered.

Outcome

Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals.

Teach it from zero

A fact table records measurable events at a declared grain and stores dimension foreign keys. Dimensions hold descriptive context used to group/filter. In this lab warehouse, `fact_sales` is one row per COMPLETED order line—not one row per order. One order can legitimately have several fact rows. Aggregating by region changes result grain to one row per region, but the regional values must reconcile to the overall fact measure.

Teacher demonstration

```
SELECT c.region, SUM(f.gross_sales) AS revenue
FROM warehouse.fact_sales f
JOIN warehouse.dim_customer c ON c.customer_key=f.customer_key
GROUP BY c.region ORDER BY c.region;
```

Guided reproduction

Write the fact grain sentence before querying. Produce revenue by region and product category; reconcile both grouped totals to overall fact revenue.

Independent task

Design a delivery fact at one row per delivered parcel line with date/customer/product-like dimensions. Identify additive measures and one measure that could be duplicated at the wrong grain.

Validate

`fact_sales` grain is correct; 12 rows, qty 30, revenue 14130.00; region/category grouped results each sum back to 14130.00.

Recover

If totals inflate, inspect join cardinality and measure grain before using DISTINCT. DISTINCT is not a repair for unexplained fan-out.

Explain back

Explain facts, dimensions and star-schema grain in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[Microsoft - Understand star schema](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-08 - Surrogate keys and referential integrity

VIDEO FIRST route

[EduFulness - Slowly Changing Dimensions Masterclass | 8:00-24:28 - keys, surrogate key, fact and dimension foundations](#)

Starting state / prerequisite

DE07-07 mastered.

Outcome

Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references.

Teach it from zero

A business key such as customer_id comes from the source/business. A surrogate key is generated inside the warehouse and has no business meaning by itself. The dimension should keep the business key for traceability while facts use the surrogate key to reference the exact warehouse row. This becomes especially valuable when one business key can have multiple historical dimension versions. Referential integrity must be proven, not assumed.

Teacher demonstration

```
SELECT customer_key, customer_id, customer_name, region FROM warehouse.dim_customer ORDER BY customer_key;
-- orphan checks should return 0
```

Guided reproduction

Display business/surrogate key pairs, then run explicit orphan checks for customer_key and product_key.

Independent task

Inside a transaction, attempt a fact row with a nonexistent product_key, save the FK error, then roll back. Explain why storing product_name as the relationship key would be weaker.

Validate

orphan customer/product refs = 0; invalid FK insert is rejected; natural key remains visible in dimension; surrogate key is not treated as user-facing meaning.

Recover

If an orphan exists, trace source key -> dimension lookup/load -> fact load. Never replace unknown keys with arbitrary existing surrogate keys.

Explain back

Explain surrogate keys and referential integrity in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Identity columns](#)

[PostgreSQL 18 - Foreign keys](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-09 - Slowly Changing Dimension Type 2 history

VIDEO FIRST route

[EduFulness - Slowly Changing Dimensions Masterclass | 36:48-42:06 SCD why/what: 1:09:05-1:40:54 SCD Type 2](#)

Starting state / prerequisite

DE07-08 mastered.

Outcome

Preserve attribute history with effective dates/current flags instead of overwriting the past.

Teach it from zero

SCD Type 2 retains multiple versions of a dimension business key. A current row is closed and a new row is inserted when a tracked attribute changes. Typical fields are `effective_from`, `effective_to` and `is_current`. A transaction keeps the close-old/open-new steps together. The toy lab deliberately teaches the pattern but does not pretend one CHECK constraint solves all production overlap/concurrency cases.

Teacher demonstration

```
-- C003 moves North -> East effective 2026-09-01
BEGIN;
UPDATE warehouse.dim_customer_scd2 SET effective_to='2026-08-31',is_current=false WHERE customer_id='C003' AND is_current=true;
-- insert new current C003 East version
COMMIT;
```

Guided reproduction

Run the SCD2 lab. Prove C003 has two versions and exactly one `is_current=true`, then narrate what history would be lost by an in-place UPDATE.

Independent task

Apply a changed future segment update to a different customer in a fresh SCD2 copy. Save before/after history and check exactly one current row.

Validate

Old version remains; new version starts on intended date; exactly one current version for changed key; effective dates do not overlap in the taught example.

Recover

If two current rows remain, stop and repair the close-old/insert-new transaction. Do not delete the old history row.

Explain back

Explain slowly changing dimension type 2 history in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[Microsoft Fabric - Slowly changing dimension Type 2](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-10 - Build and validate a small analytical database

VIDEO FIRST route

[Kahan Data Solutions - Data Modeling Tutorial: Star Schema | 0:31-14:55 - quick model-design refresh only: do NOT copy a project solution](#)

Starting state / prerequisite

DE07-01 through DE07-09 mastered.

Outcome

Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation.

Teach it from zero

A successful CREATE/INSERT run is not enough. A reliable build has objective invariants: declared grain, row counts, eligibility/exclusion rules, unique business grain, zero orphans and monetary/quantity totals that reconcile. The setup is intentionally rerunnable at the module level; validation must be rerun after any material model change.

Teacher demonstration

```
SELECT COUNT(*) AS fact_rows, SUM(qty) AS total_qty, SUM(gross_sales) AS total_revenue FROM warehouse.fact_sales;  
-- Expected baseline: 12, 30, 14130.00
```

Guided reproduction

Run the canonical setup/build then final validation. Explain every control: 12 fact rows, qty 30, revenue 14130.00, cancelled O1005 contributes zero, no duplicate (order_id,line_no), zero orphans.

Independent task

From a clean stage07_de rebuild, execute the build and validation without looking at the teacher SQL except when repairing. Save a validation transcript.

Validate

All baseline controls pass and region totals East 3710, North 2390, South 1750, West 6280 sum to 14130.

Recover

If one control fails, isolate eligibility -> dimension lookup -> fact grain -> measure calculation. Do not adjust expected totals to match a broken load.

Explain back

Explain build and validate a small analytical database in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 documentation](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-11 - Bulk loading with COPY / staging boundaries

VIDEO FIRST route

[vlogize - Importing CSV Files into PostgreSQL Tables | Whole focused video - COPY vs client-side \copy/file-location problem: Teaching Book controls exact course method](#)

Starting state / prerequisite

DE07-10 mastered.

Outcome

Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy.

Teach it from zero

COPY is a PostgreSQL SQL command; when it names a file, the server process reads/writes from the server environment and is constrained by server-side permissions. psql \copy is a psql meta-command: the client reads/writes the local file and streams data through the connection. In a learner workstation this distinction explains why a local path can work with \copy but fail with server-side COPY. Staging is a receiving area: load permissively enough to inspect, reconcile source business rows, then transform into final constrained tables.

Teacher demonstration

```
CREATE TABLE source.stg_customers(customer_id text,customer_name text,region text,segment text);
-- psql client-side:
-- \copy source.stg_customers FROM 'customers_stage.csv' WITH (FORMAT csv, HEADER true);
SELECT COUNT(*) FROM source.stg_customers;
```

Guided reproduction

Load the bundled customers_stage.csv into source.stg_customers using psql \copy or documented pgAdmin Import/Export. Save source business-row count and staging COUNT(*).

Independent task

Load carriers_stage.csv to a new staging table, reconcile rows, flag one malformed/unknown region row before any final constrained insert.

Validate

Source and staging business-row counts reconcile; header is not counted; learner explains COPY vs \copy and staging vs final tables.

Recover

If a local path fails with COPY, do not change file permissions blindly. First determine whether the command is server-side COPY or client-side \copy.

Explain back

Explain bulk loading with copy / staging boundaries in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - COPY](#)

[PostgreSQL 18 - psql \copy](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-12 - Schemas, roles and least-privilege boundaries

VIDEO FIRST route

[Microsoft Developer - Roles, Privileges, and Security in PostgreSQL | 13:56-21:57 privileges + principle of least privilege; optional 21:57 onward demo](#)

Starting state / prerequisite

DE07-11 mastered.

Outcome

Design schema/role boundaries that grant only the operations a workload needs.

Teach it from zero

PostgreSQL roles can be login identities or group-like privilege containers. Database CONNECT, schema USAGE/CREATE and table SELECT/INSERT/UPDATE/DELETE are different powers. Object ownership is stronger than ordinary grants. A reporting workload normally should not run as the database owner or superuser. Least privilege means writing down what the workload needs and explicitly what it does not need. Default privileges/future objects are a separate consideration from existing tables.

Teacher demonstration

```
CREATE ROLE stage07_reader NOLOGIN;  
GRANT USAGE ON SCHEMA warehouse TO stage07_reader;  
GRANT SELECT ON ALL TABLES IN SCHEMA warehouse TO stage07_reader;
```

Guided reproduction

Design warehouse_reader and source_loader privilege matrices. If your local account is allowed, create only a NOLOGIN reader and inspect grants; otherwise do the design evidence without changing system privileges.

Independent task

For a fresh reporting service, state exact database/schema/table privileges, owner boundary and how future-table access would be handled. Reject GRANT ALL as an unexplained shortcut.

Validate

Reader has no write/drop/ownership requirement; loader does not receive warehouse-owner powers; design distinguishes schema USAGE from table privileges.

Recover

If permission errors appear, inspect current_user, ownership and actual grants. Do not switch the application to postgres/superuser as the repair.

Explain back

Explain schemas, roles and least-privilege boundaries in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - Privileges](#)

[PostgreSQL 18 - Schemas and privileges](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.

DE07-13 - Schema change, backup/restore and handoff validation

VIDEO FIRST route

[WittCode - Backing Up a Postgres Database with pg_dump and pg_restore | 0:34-6:06 - pg_dump, formats, pg_restore and restore demonstration](#)

Starting state / prerequisite

DE07-12 mastered.

Outcome

Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy.

Teach it from zero

Schema change should be scripted, reviewed and validated rather than hidden in GUI clicks. `pg_dump` creates a logical export; custom format (-Fc) is restored with `pg_restore` and supports selective restore features. A backup file is not recovery proof until a restore to a separate database/environment succeeds and the same controls pass. Never place a real password in the command text or evidence. A handoff must include build/migration scripts, grains, control totals, role assumptions, backup metadata, restore validation and known limitations.

Teacher demonstration

```
ALTER TABLE source.products ADD COLUMN IF NOT EXISTS active boolean NOT NULL DEFAULT true;
-- terminal examples:
-- pg_dump -Fc -d stage07_de -f stage07_de.backup
-- createdb stage07_restore_test
-- pg_restore -d stage07_restore_test stage07_de.backup
```

Guided reproduction

Apply the active-column migration and prove its definition. If `pg_dump/pg_restore` are available, dump and restore to `stage07_restore_test`, then rerun final controls.

Independent task

Write a handoff checklist and a changed reversible migration for a new nullable source attribute. Include rollback/recovery thought, validation and restore target separation.

Validate

Migration is scripted; backup metadata recorded; restore target is separate; post-restore controls match baseline where runtime is available. If runtime is unavailable, limitation is recorded—not guessed.

Recover

If restore fails, keep original database untouched, inspect dump format/version/target/permissions, and retry only against the separate test target.

Explain back

Explain schema change, backup/restore and handoff validation in 60-90 seconds: what problem it solves, one failure mode, and the exact evidence you would inspect before claiming success.

Current authoritative reference(s)

[PostgreSQL 18 - pg_dump](#)

[PostgreSQL 18 - pg_restore](#)

Master only when you can reproduce the skill on changed data without reopening Review. A correct-looking result without validation evidence is not mastery.